AWI-ESM3 on albedo: why the coupled runs are slow, and how to make them fast

Jan Streffing

Last updated: 2026-10-16

Abstract

The AWI-ESM3 CMIP7 spin-up (TCO95L91 OpenIFS, CORE3 FESOM2 with cavities, LPJ-GUESS, XIOS, 47 nodes) moved from levante to albedo on 2026-09-26 and ran there at 49 to 57 SYPD, against 83 SYPD on levante. This report follows why, and how the layout was changed until the model ran faster on fewer nodes. Where we are (2026-09-29): 25 nodes step 2.40 s per model day (about 99 SYPD in steady state, 6.1 node-hours per model year against 19.8 in the first albedo legs), with FESOM at 1800 s and without OpenMP threads, its 768 ranks spread 96 per node, XIOS and the runoff mapper on the OpenIFS nodes, and OpenIFS and FESOM now sharing the critical path. With the carbon cycle (REcoM, 33 FESOM tracers) the same layout is 6.8 times slower; there FESOM's rank count is the lever, and 2688 FESOM ranks on 36 nodes run at 46 to 52 SYPD for 16.6 to 18.8 node-hours per model year, against 14.6 SYPD and 41.2 on 25 nodes. The steps are in Section 1, the current state and open questions in Section 2, falsified explanations in Section 3.

1 The campaign, in the order it happened

1.1 First albedo legs (2026-09-26–27)

After the OASIS partition fix and the OpenIFS cold-start fix, the first clean 10-year legs on albedo took 4 h 20 min to 4 h 40 min of wall time (jobs 47456102, 47472034, 47472035). Levante had done the same legs in about 3 h. Four other legs died in an XIOS server with a SIGBUS; that problem is documented separately (`xios_sigbus_albedo_report_2026-09-28.md`) and is not part of the speed question.

1.2 Albedo against levante, same configuration (2026-09-28)

We compared the finished ccnice leg 2110–2119 on albedo (job 47472034) with levante's leg 2090–2099 (job 27681023). Both used 47 nodes with FESOM 1792×1, OpenIFS 384×4, LPJ-GUESS 64×8, XIOS 16×128, identical namelists (`whichEVP`=1, $\alpha = \beta = 300$, `linfs`, cavities) and single-precision OpenIFS on both machines.

	levante	albedo	ratio
Leg wall time (s)	10385	15483	1.49
OpenIFS wall per step (s)	0.118	0.173	1.47
OpenIFS CPU per step, 4 threads (s)	0.452	0.548	1.21
FESOM ocean (s per leg)	4406	5694	1.29
vertical mixing	1111	1304	1.17
tracer advection	933	1128	1.21
ssh solver	1314	1850	1.41
FESOM ice EVP	2725	4992	1.83
FESOM waiting for the atmosphere (`ice read forc`)	441	1200	
FESOM output (means and XIOS send)	1925	1831	0.95

Table 1: Timings per 10-year leg. FESOM numbers are rank-0 means from its `MODEL RUNTIME per task` block; OpenIFS from `ifs.stat` time stamps.

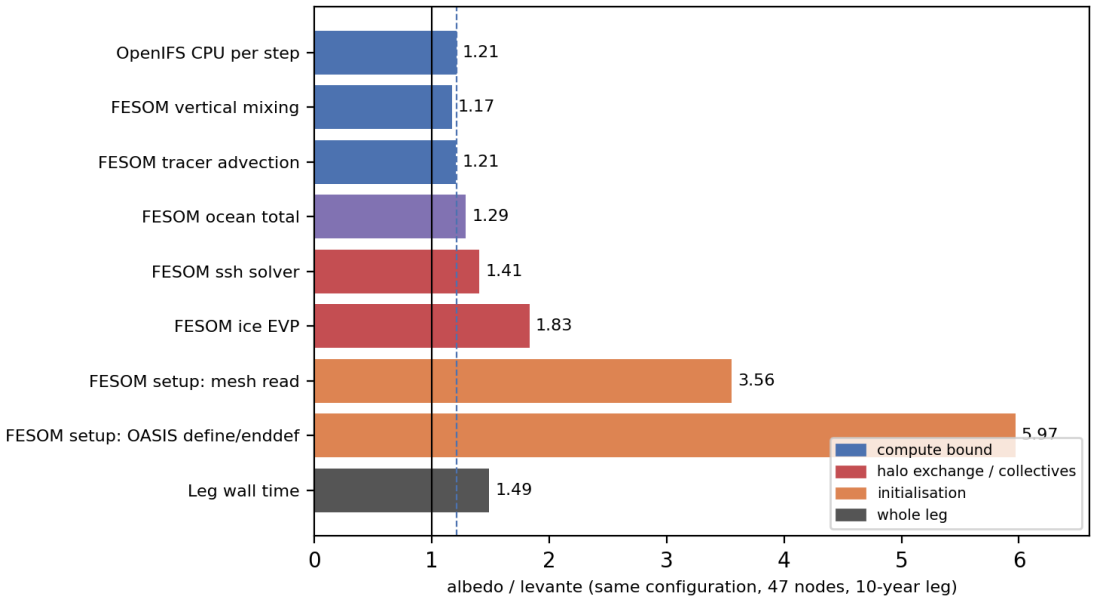


Figure 1: Albedo over levante for the same configuration. Compute-bound parts (blue) slow down by about 20 %, the dashed line; a cache-resident benchmark kernel measures levante's Zen3 cores (EPYC 7763, 2449 MHz under full load) as 16 % faster than albedo's Zen2 cores (EPYC 7702, 2331 MHz). Parts dominated by halo exchanges and collectives (red) slow down by 41 to 83 %, and the communication-heavy OASIS setup (orange) by a factor of six. The whole leg is 1.49 times slower.

The build is not the cause: both machines use the Intel compiler (ifort 2021.6 on albedo, 2021.5 on levante) with the same base flags, and albedo adds `-march=core-avx2 -qopt-malloc-options=2 -qopt-prefetch=5 -unroll-aggressive`, which levante does not have. Both machines have HDR100 InfiniBand.

1.3 Load balance on albedo from logs (2026-09-28)

This first estimate is superseded by LUCIA below. OpenIFS spends 0.143 s of CPU per step on its 4 threads (0.571 over 4) out of a 0.173 s step, so it computes 83 % of the time. FESOM is busy 98.5 % of the wall time when its `diag` (2.8 %) and `output` (12.1 %) categories are counted, of which 7.9 % is the wait for the atmosphere. Both models are saturated and FESOM is the marginally slower one. The daily LPJ-GUESS exchange costs 1.7 % (the step after it takes 0.242 s against 0.17 s), and there are no year-end stalls: every model year takes 1471 to 1645 s.

1.4 Comparison with Miguel's 100-SYPD runs (2026-09-28)

The albedo sheet of `AWIESM3-Performance.ods` lists 100.26 SYPD for FESOM 1792 × 2, OpenIFS 384 × 4 on CORE3Cav, measured over steps 3–1416. Over the same window our leg runs at 53.2 SYPD. Two differences account for the gap:

- CMIP7 output. OpenIFS FullPos (`DYNFPOS` 16.9 %, `GRIDFPOS` 4.7 %, `SU4FPOS` 2.9 %) is 24.5 % of OpenIFS CPU for 209 files and 42 GB per model year. Without it the step would be about 0.108 s. This output is required.
- FESOM at one thread. Miguel ran FESOM with two OpenMP threads, twice our FESOM cores, which removes the wait of OpenIFS on the ocean.

Miguel's run directories are not readable for us (mode 700), so FESOM's own timing in his runs is unknown.

1.5 XIOS is not a bottleneck (2026-09-28)

From the XIOS performance reports of the finished leg: the eight first-level servers are busy 2.6 to 2.8 % of the time, the eight second-level (file-writing) servers 34 to 41 %. The client ratio is 0.74 % on OpenIFS rank 0 and 0.04 % on FESOM rank 0. XIOS occupies 16 of 47 nodes with one server each, which is the one clear waste of node hours.

1.6 Initialisation is five times slower (2026-09-28)

A 1-day test leg (TEST_xios1d) takes 9 min: 5.5 min initialisation, 57 s of integration and 2.5 min for finalisation, tidy and the next leg's preparation. File movement is threaded (127 threads) and takes about 20 s. FESOM's setup timer is the dominant part:

FESOM setup on rank 0 (s)	levante (3 legs)	albedo (4 legs)	ratio
mesh and partition read	18–21	66–70	3.5
restart read	7–8	20–40	2.5
other: OASIS define and `oasis_enddef`	37–38	223–234	6
total	64–68	313–335	5

Table 2: The file reads slow down like GPFS I/O; the OASIS router setup, an all-to-all between 1792 and 384 ranks, slows down like the other communication-heavy parts. `oasis_enddef` also waits for the slowest model to finish its own initialisation, so part of it may be LPJ-GUESS or OpenIFS input on GPFS.

1.7 Node-to-node spread (2026-09-28)

The ccnice leg 2120–2129 (job 47473364) ran at 0.202 s per OpenIFS step (48.9 SYPD) against 0.173 s in the leg before, with the same configuration on a different set of nodes.

1.8 TEST_perfA: FESOM with two threads (2026-09-28)

A 2-month leg with FESOM 1792 × 2 (61 nodes) ran at 0.149 s per OpenIFS step (66.4 SYPD) against 0.176 s (56.0 SYPD) with one thread. FESOM's compute per OpenIFS step fell only 8 % (0.145 to 0.134 s): the EVP by 23 %, the ocean by 7 %, while the ssh solver got 20 % slower. Doubling FESOM's cores therefore buys little; FESOM at 1792 ranks is limited by communication, not by arithmetic. (Hardware counters later showed that memory bandwidth is not the limit either, Table 15.) FESOM's setup took 133 s in this run against 313 to 335 s in all earlier albedo runs (mesh read 22 against 66–72 s), with the same files on the same filesystem, so the initialisation cost varies from job to job.

1.9 LUCIA: FESOM is the limiter (2026-09-28)

With `oasis3mct.use_lucia: true` OASIS3-MCT 5 writes `load_balancing_info.txt` and one `timeline_<component>.nc` per component into the work directory. For TEST_perfA:

component	computing (s)	waiting (s)
FESOM	190.7	19.2
OpenIFS	92.8	125.4
LPJ-GUESS	≈ 0	300.9

Table 3: 2-month leg, 1416 OpenIFS steps. The OpenIFS numbers of this summary turned out to be wrong; see the timeline analysis later in this section, which gives 186 s computing and 54 s waiting. The CPU column of `ifs.stat` counts MPI busy-waiting as CPU time, which is why the log-based estimate above put OpenIFS at 83 %. The SYPD line of the LUCIA summary is broken (*****); the compute and wait columns add up to the wall time and are sound.

1.10 What changed since August (2026-09-28)

The August albedo runs under `/albedo/work/user/jstreffi/runtime/awiesm3-develop/` (awiesm3sp_lucia and relatives, 1 month each, FESOM 1792×1 on the same CORE3 mesh with cavities) used a different FESOM configuration:

	August (awiesm3sp_lucia)	now (PICAL_ccnice)
time step	1800 s (48 per day)	1200 s (72 per day)
vertical mixing	KPP	cvmix TKE + IDEMIX
`K_GM_max`	1000	2500
FESOM compute per model hour	0.073 s	0.145 s
per FESOM step	0.037 s	0.048 s

Table 4: Same mesh, EVP settings (mEVP, 120 subcycles), ALE and rank count. FESOM now takes 1.5 times as many steps per day, each 32 % more expensive, which doubles its cost per model day. Since FESOM is the limiter, this accounts for most of the distance to the 100-SYPD runs in the performance sheet.

The August runs also show FESOM setup times between 14 and 274 s for identical configurations, which confirms that the initialisation cost is not a property of the configuration.

1.11 The MPI environment differs between the machines (2026-09-28)

esm_tools sets a tuned MPI environment on levante and almost none on albedo:

	levante (production job)	albedo (production job)
point-to-point layer	`OMPI_MCA_pml=ucx`, `btl=self`	OpenMPI default
collectives	HCOLL (`coll_hcoll_enable=1`, multicast)	OpenMPI `tuned`
transports	`UCX_TLS=mm,knem,cma,dc_mlx5,dc_x,self`, unified mode	UCX default
OpenMP runtime	`KMP_AFFINITY=scatter`, `KMP_LIBRARY=turnaround`	nothing
kernel	`mitigations=off`	all speculative-execution mitigations o

None of albedo's OpenMPI builds (4.1.3, 4.1.5, 4.1.6) contains the HCOLL component, although `libhcoll` is installed under `/opt/mellanox/hcoll`. FESOM's own `env/albedo/shell` builds with Intel MPI instead.

1.12 MPI and node benchmarks, albedo against levante (2026-09-28)

A small MPI benchmark (`bench/mpibench.c`) was compiled with each machine's model MPI and run with 128 ranks per node:

	albedo, default	albedo, best setting	levante, production env
core speed, cache-resident kernel (GFlop/s per rank)	5.53		6.41
memory bandwidth, triad (GB/s per rank)	2.48		2.59
ping-pong 8 B between nodes (μs)	1.6	1.4	3.4
allreduce 8 B, 1 node (μs)	7.8		7.1
allreduce 8 B, 16 nodes (μs)	50–88	21 (DC random)	30
halo 6×16 KiB, 1 node (μs)	114	114 (default is best)	59
halo 6×2 KiB, 16 nodes (μs)	164	23.5 (DC random)	
halo 6×16 KiB, 16 nodes (μs)	287–343	148 (DC random)	74

Table 5: Per-core speed differs by 16 % and memory bandwidth by 4 %. Inside one node the halo exchange is twice as slow on albedo whatever transport is chosen (default, levante's `mm,knem,cma`, posix, sysv, CMA, knem, no zero-copy, OpenMPI vader without UCX); the default is already the fastest. Across nodes, UCX's DC transport with the default send policy `dcs_quota` is slow; `UCX_DC_MLX5_TX_POLICY=rand` makes collectives and small halos faster than levante. More DC initiators (16, 32, 64) or unified mode do not help. The benchmark runs used one leaf switch.

1.13 Model tests of the MPI settings and the time step (2026-09-28)

All tests are 2-month legs of the crunveg configuration on 47 nodes with FESOM 1792×1 and LUCIA on, run on `paleodyn.paleodyn`; the speed is measured over OpenIFS steps 3–1416.

test	MPI setting	FESOM step	s per OpenIFS step	SYPD
baseline (TEST_xiosbus)	default	1200 s	0.176	56.0
TEST_perfE (RC)	`UCX_TLS=rc_x,sm,self`	1200 s	fails at start-up	
TEST_perfE	`coll/han`	1200 s	0.184	53.6
TEST_perfG	DC, random policy	1200 s	0.188	52.4
TEST_perfF	`coll/han`	1800 s	0.137	71.9
TEST_perfH	DC, random policy	1800 s	0.136	72.6
levante production	levante env	1200 s	0.118	83.3

Table 6: The RC transport dies during initialisation with `ud_ep.c:280 Fatal: UD endpoint ... unhandled timeout error`: the OASIS all-to-all needs a connection per rank pair among 2256 ranks. Neither `coll/han` nor the DC random policy changes the model speed, although both make the benchmark collectives 2 to 7 times faster. Only the time step does.

FESOM per OpenIFS step (s)	baseline	`coll/han`	DC random
ice EVP	0.0576	0.0613	0.0558
ssh solver	0.0215	0.0230	0.0226
vertical mixing	0.0147	0.0148	0.0179
FESOM total (ice+ocean)	0.145	0.156	0.160

Table 7: The communication-heavy EVP and ssh solver do not move with the MPI settings. Even the purely local vertical mixing varies by 20 % between runs, which points at node-to-node variation. LUCIA shows FESOM computing 96 to 99 % of the time in all tests, so OpenIFS does not mask a FESOM gain.

At 1800 s FESOM costs 0.118 s per OpenIFS step instead of 0.156, the EVP 0.036 instead of 0.061. Both production runscripts were switched to 1800 s. The OASIS lags are all 3600 s and do

not depend on it.

1.14 Earlier FESOM reports and the build (2026-09-28)

The FESOM tracker holds related albedo reports: #874 (FESOM 2.7 sea ice slow on every node on albedo, which went away after an albedo system update), #885 and #400 (`ENABLE_ALBEDO_INTELMPI_WORKAROUND` barriers in the native netCDF output gather against an Intel MPI message-queue pile-up; not relevant with OpenMPI and XIOS, and not in our branch), #1044/#1048 (barriers around the `sigma_xy` exchange cost 3–5 %; the fix is in our branch), #395 (`-mtune=core-avx2` for Zen, not set in our build) and #685 (GCC was twice as slow as Intel on levante; both our builds use Intel). Our build has the asynchronous I/O threads off (`ASYNCHRONOUS_IO_THREADS=OFF`), so no extra thread competes with the pinned FESOM ranks.

1.15 Profiling FESOM on albedo (2026-09-28)

VTune 2022.3 cannot run for normal users on albedo: `kernel.yama.ptrace_scope` is 2 and `perf_event_paranoid` is 2, and no sampling driver is loaded. We therefore wrote a small program-counter sampler (`bench/pcsampler.c`) that is preloaded into FESOM ranks 0, 896 and 1791 through `bin/fesom_prof.sh`. It samples the interrupted instruction pointer on a CPU-time timer (10 ms resolution at the kernel's 100 Hz tick) and writes a histogram and the memory map at exit; `bench/pcs_report.py` turns them into time per library (FESOM, MPI, UCX, libc) and per FESOM routine. TEST_prof (2 months, 1800 s, default MPI) runs it. The first run sampled the whole job and was dominated by the initialisation, so the sampler now bins samples in 10 s intervals and the report is restricted to the time-stepping window taken from the `ifs.stat` time stamps. A wrapper for FESOM has to go through `fesom.execution_command`, because `bin_in_work` names the staged FESOM binary after `executable`.

The profile of the stepping window (250 s, about 2×10^5 samples per rank, job 47476015) is in Table 8. Two thirds of FESOM's wall time is spent inside MPI. The UCX libraries are stripped, so their samples carry the name of the nearest exported symbol (`uct_dc_mlx5_ep_is_connected`, `uct_mm_iface_flush`); the code behind them is UCX's static progress loops, which means FESOM ranks spend this time polling for messages, not doing work. The compute share differs between ranks: rank 896 computes 24 % of the time and has almost no sea ice (EVP 0.5 %), while ranks 0 and 1791 carry ice (EVP 4 to 5 %) and compute 30 to 35 %. The waiting is therefore at least partly the ice load imbalance, which the slower halo exchange on albedo makes more expensive. The accumulation of output means (`io_meandata::update_means`) costs 7 % of FESOM's wall time on every rank, more than EVP and IDEMIX together and about a quarter of FESOM's own computing; it does not depend on the machine. Thread-safety overhead of `MPI_THREAD_MULTIPLE` (`pthread_spin_lock`, `pthread_self`) is 8 to 10 %, but it sits inside the polling loop and would largely turn into more polling if removed.

	albedo (265 s)			levante (215 s)		
share of wall time (%)	r0	r896	r1791	r0	r896	r1791
FESOM code (`libfesom`, `fesom`, libc)	30.0	24.1	34.9	26.6	23.1	25.1
MPI, UCX and pthread (polling)	65.9	74.2	60.1	69.7	75.4	72.1
OpenMP runtime	3.9	1.7	4.9	3.5	1.3	2.7
`io_meandata::update_means`	7.5	7.1	7.0	7.2	7.0	7.1
`evpdynamics`	4.1	0.5	4.8	3.1	0.5	2.1
IDEMIX	1.9	1.8	2.6	1.8	1.7	1.8
`pthread_spin_lock` + `pthread_self`	6.1	10.2	7.6	5.3	6.1	6.4
FESOM code, seconds	79.5	63.9	92.5	57.2	49.7	54.0
MPI polling, seconds	174.6	196.6	159.3	149.9	162.1	155.0

Table 8: Program-counter profile of three FESOM ranks during the time stepping of the same 2-month run on both machines (TEST_prof, 2026-09-28, albedo job 47476015, levante job 27762013; stepping time in the header). The shape is the same on both machines: FESOM ranks compute a quarter to a third of the time and poll MPI for the rest, and the ice-free rank 896 waits longest. Albedo is 1.2 times slower in the polling and 1.1 to 1.7 times slower in FESOM's own code.

The same run on levante (2 months from the same PICAL_ccnice restart, FESOM 1800 s, levante's default MPI environment and its leadclose FESOM binary) steps in 215 s against albedo's 265 s, a factor of 1.23. Its profile has the same shape (Table 8): FESOM ranks poll MPI for 70 to 75 % of the time on levante too, `update_means` costs 7 % on both, and the thread-locking share is similar (5 to 6 % on levante). Albedo's MPI is therefore not pathologically slow; FESOM at 1792 ranks is communication-bound on both machines, and albedo loses about 20 % in the communication and 10 to 70 % in FESOM's own code. The largest code difference is on rank 1791, where EVP takes 12.7 s on albedo against 4.5 s on levante; the two FESOM binaries differ (albedo: main with PRs #1060 and #1087; levante: leadclose), so this part is not yet a pure machine comparison. The albedo sampler dropped 9 % of its samples when its hash table filled, levante's under 1 %; the dropped samples fall preferentially on code with many distinct addresses, so albedo's FESOM-code share is if anything underestimated. Both runs carry the sampler and LUCIA overhead; they are slower than production (albedo 53 against 72 SYPD at 1800 s) and are only compared with each other.

Why the factor is 1.23 here and 1.49 for the 10-year legs. Table 10 gives the stepping time of the first two months (1416 OpenIFS steps) in every run. Split by model day (1 s resolution of `ifs.stat`), the two TEST_prof runs step equally fast between day 2 and day 58: 3.37 s per model day on albedo, 3.35 on levante. The whole difference sits in two days. The first day, with the cold-started OpenIFS, takes 31 s on albedo against 16 s; the last day, when the leg writes its restarts and closes its output, takes 37 s against 7 s. In the 10-year legs at 1200 s the steady state itself differs: 4.16 s per model day on albedo (1517 s per year) against 2.84 on levante (1036 s per year), a factor of 1.46. LUCIA on levante (TEST_prof, 1800 s) shows FESOM computing 178.7 s and waiting 29.5 s, OpenIFS computing 149.1 s and waiting 63.1 s: FESOM is the limiter on levante as well. Albedo's OpenIFS line is again negative and cannot be used. The sampler slows the run (albedo 3.37 s per model day with it against 3.03 in perfH without it), and it slows the three sampled FESOM ranks on which all others wait, so it may equalise the two machines artificially; a levante run at 1800 s without the sampler settles it.

That run, TEST_1800 (levante job 27762423, TEST_prof without the sampler), steps 2.77 s per model day between day 2 and day 58, against 3.03 for albedo's perfH: a steady-state factor of 1.09 at 1800 s, and 1.15 over the two months including the first and last day (182 against 210 s). Levante gains only 2.5 % from the longer FESOM step (2.84 s per model day at 1200 s in the 2090 leg). LUCIA explains why: on levante at 1800 s FESOM computes 146 s and OpenIFS 149 s, both wait about 31 s, so the two components are balanced and OpenIFS now sets the pace as much as FESOM does. On albedo the LUCIA summary suggested FESOM still limits at 1800 s; the timelines below show that it does not. The 1.46 of the 10-year legs is therefore a property of the 1200 s FESOM step on albedo; at 1800 s the remaining machine factor is about 1.1 in steady state.

1.16 LUCIA timelines: at 1800 s both components are balanced on both machines (2026-09-28)

The LUCIA summary cannot be trusted for OpenIFS: its computing time is negative in most runs. The per-rank event timelines (`timeline_<component>.nc`) can: for every rank they hold the start and end of each `GET` and `PUT`. `bench/lucia_timeline.py` sums the time spent inside `GET` after the first coupling exchange (whose `GET` absorbs the other component's initialisation) and counts the rest of the window as computing.

run	FESOM step	FESOM computing	FESOM waiting	OpenIFS computing	OpenIFS w
albedo perfE (han)	1200 s	294.0	2.9	203.5	
albedo perfG (DC random)	1200 s	290.7	10.4	201.8	
albedo perfF (han)	1800 s	214.3	16.3	202.9	
albedo perfH (DC random)	1800 s	208.8	19.8	209.3	
levante TEST_1800	1800 s	167.3	32.1	170.7	

Table 9: Seconds per 2-month leg, mean over ranks, from the LUCIA timelines. At 1200 s FESOM limits on albedo and OpenIFS waits 115 s. At 1800 s the two compute equally long on both machines, and the remaining waits on both sides come from steps in which one or the other is slower. Albedo is 1.22 times slower than levante in OpenIFS and 1.25 times in FESOM.

A faster FESOM alone therefore buys little at 1800 s: at most the time OpenIFS spends waiting (37 to 48 s of about 250 s), and in practice less, because in the other steps OpenIFS is the slower one. Further speed needs both components to become faster, or more cores for OpenIFS.

run	FESOM step	1 day	10 days	30 days	59 days
albedo ccnice leg 2110	1200 s	33	86	167	283
albedo ccnice leg 2120	1200 s	33	80	173	307
albedo perfE (han)	1200 s	33	71	155	291
albedo perfG (DC random)	1200 s	30	68	154	285
albedo perfF (han)	1800 s	32	61	122	233
albedo perfH (DC random)	1800 s	29	54	117	210
albedo TEST_prof (sampler)	1800 s	31	61	127	260
levante ccnice leg 2080	1200 s	15	39	102	212
levante ccnice leg 2090	1200 s	14	39	92	177
levante TEST_prof (sampler)	1800 s	16	42	107	214
levante TEST_1800	1800 s	15	43	99	182

Table 10: Cumulative stepping time in seconds after 1, 10, 30 and 59 model days, from the `ifs.stat` time stamps. The first model day (OpenIFS cold start) costs twice as long on albedo in every run.

1.17 FESOM main with the OpenMP performance stack (2026-09-29)

FESOM PRs #1081–#1085 and #1093–#1095 (node-owned gathers instead of locked scatters, `MPI_THREAD_FUNNELED`, no lock calls at one thread, EVP halo overlap) were merged into `main` on 2026-09-29. Neither machine's coupled runs had them: albedo runs `leadclose-1087` (8de217d6), levante `fesom2-leadclose` (1753f0bf). `main` was merged into PR #1060 (`use_atm_ice_tskin`, which production uses), pushed as 1ef85546, and built with the production options. #1095 itself changes only `EVPdynamics` (`whichEVP`=0); we run mEVP (`EVPdynamics_m`), which #1094 does cover. Two 2-month tests repeat perfH (UCX DC random, FESOM 1800 s, LUCIA) with the new binary. perfJ runs FESOM at 1024 ranks × 2 threads on 16 nodes; its OASIS set is production's (levante `dist_1792` order) reordered to `dist_1024` with `AWI-ESM/oasis_reorder_tool`, including grids, masks and areas, which the `CONSERV` couplings read. The order was checked against `nod2d.out` and the `my_list` files of both partitions.

run	FESOM	s per model day	FESOM computing	OpenIFS computing	FES
perfH, leadclose 8de217d6	1792×1, 14 nodes	3.04	208.8	209.3	
perfI, main + #1060	1792×1, 14 nodes	2.72	182.2	200.6	
perfJ, main + #1060	1024×2, 16 nodes	2.74	160.7	177.6	
levante TEST_1800, leadclose	1792×1, 14 nodes	2.77	167.3	170.7	

Table 11: Steady-state stepping (days 2–58) and LUCIA timeline seconds per 2-month leg. The new FESOM computes 13 % less at the same layout and 23 % less at 1024×2; albedo now steps as fast as levante did with the old binary. FESOM waits for OpenIFS in both new runs, so the coupled speed is set by OpenIFS; its computing time varies by about 10 % between runs on different nodes.

SST stays within 31.6 to 32.2 °C maximum and −2.0 °C minimum in both runs. After two months the root-mean-square SST difference is 0.67 K between perfH and perfI (same partition, different binary) and 0.73 K between perfI and perfJ (same binary, different partition), the size of the chaotic divergence; the global means agree within 0.08 K. The reordered coupling set therefore works end to end.

1.18 Fewer nodes: XIOS packed, FESOM and OpenIFS with more threads (2026-09-29)

XIOS is MPI-only; its 128 threads per server only reserved whole nodes for memory. The largest XIOS server reaches 44 to 46 GB in a 10-year leg (task 2249 in all three production legs) and 8.6 GB in two months, so four servers fit on a 257 GB node of their own (next to OpenIFS ranks only two do, see the production layout below). Rank thread counts are kept at 2, 4 or 8, because albedo's cores share L3 in groups of 4 and a 3-thread rank would straddle groups. New node-aware FESOM partitions `dist_384` ($n_part = 12, 32$) and `dist_768` $(12, 64)$ were made with `fesom_meshpart` in a private copy of the mesh, and the coupling set was reordered to each with the same checks as for 1024. All runs use the new FESOM binary.

run	layout	nodes	s per model day	node·s per model day	FESOM computing	OpenIFS
perfI	reference	47	2.72	128	182.2	
perfK	XIOS 16×32	35	2.86	100	199.5	
perfL	perfK + FESOM 384×4	33	2.95	97	188.9	
perfM	perfK + FESOM 768×2	33	2.86	94	179.6	
perfN	perfK + OpenIFS 192×8	35	2.98	104	199.7	

Table 12: Steady-state stepping (days 2–58) and LUCIA timeline seconds per 2-month leg. perfM needs 26 % fewer node-seconds per model day than perfI at 5 % lower speed. OpenIFS at 192×8 is slower than at 384×4. perfK is bit-identical to perfI, so its difference is placement and node noise only; OpenIFS computing time varies from 168 to 210 s between runs with the same OpenIFS layout, so single-run differences of 5 % are not significant.

XIOS servers were at most 15 % busy in perfK (24 % in perfI). SST stays within −2.1 and 32.7 °C in all runs, and the runs with new partitions differ from perfI by 0.64 to 0.72 K RMS after two months, the chaotic spread. The XIOS memory growth over a full leg is not covered by two-month tests; a 10-year leg in the packed layout has to confirm that no server exceeds 64 GB.

1.19 Sharing nodes: XIOS and rnfmap on the OpenIFS nodes (2026-09-29)

esm_tools' `taskset` launcher decided placement in two places that had to agree: the hostlist came from a running core counter in the job script, while `prog_<model>.sh` pinned each rank to (rank × threads) mod 128. The two only agree while every component boundary falls on a node edge, so no two components could share a node, and XIOS and the one-rank river-runoff mapper each held whole nodes. A new placement module computes one (node, first core) pair per global rank,

writes it to `work/rank_placement`, and both the hostlist and every `prog_<model>.sh` read it (esm-tools/esm_tools#1603, branch `feat/awiesm3-v3.4-co2`). A component with `interleave_into: [oifs]` takes cores at the end of OpenIFS nodes, `ranks_per_node` of them per node, and only on as many nodes as it needs. Every rank starts on a multiple of its thread count and never crosses a node, which keeps 1-, 2-, 4- and 8-thread ranks inside one 4-core L3 group.

run	layout	nodes	s per model day
perfM	FESOM 768×2, XIOS 16×32 on own nodes, old placement	33	2.86
placeA	the same through the placement table	33	2.79
placeB	XIOS 16×4 four per OpenIFS node, rnfmap on an OpenIFS node	29	2.79

Table 13: Steady-state stepping, days 2–58. Sharing nodes frees four nodes at unchanged speed. placeA reproduces perfM's node and core sets; its hostlist is byte-identical to the one the old code wrote.

1.20 Reproducibility and the lock-free sea-ice solvers (2026-09-29)

perfM and placeA use the same binary, partition, thread counts and inputs, and differ only in the nodes they ran on and the core order of the LPJ-GUESS and XIOS blocks, yet their output differs from the first model day. The build has `OPENMP_REPRODUCIBLE=OFF`, and the modified EVP solver we run (`whichEVP`=1, `ice_maEVP.F90`) still accumulated element contributions into nodes under OpenMP locks, so the summation order followed thread timing. FESOM's thread-invariance CI test runs `whichEVP`=0 and never exercised mEVP. A port of mEVP to node-owned gathers (each node sums its own elements in a fixed order) made two identical runs at 768×2 (mevpB1, mevpB2) bit-identical over all 59 days, and FESOM computed 157–161 s against 179 s in placeB.

The same day FESOM PRs #1097 (mEVP and aEVP gathers, with CI variants for both), #1098 (split-explicit barotropic gathers and a restart fix for `d_eta`) and #1099 (diagnostics and IDEMIX2 gathers, the lock array `partit%plock` removed, barriers before collectives dropped) were reviewed and merged. They make the port redundant; `main` was merged into #1060 again (53e095ec) and that build used from here on.

runs	what differs	result	s per model day
nolockB1 / nolockB2	nothing (rerun, 768×2)	bit-identical, 59 days	2.65 / 2.63
nolockB1 / nolockB4	FESOM 2 against 1 thread, same cores	differ from day 1	2.65 / 2.51
perfI / nolockI	1ef85546 against 53e095ec, 1792×1	differ from day 1	2.72 / 2.79

Table 14: Bit-comparison of daily FESOM output (SST, sea-ice concentration and velocity, SSH). A rerun at a fixed thread count reproduces exactly; a different thread count does not. Every field differs after one day, including the atmospheric stress over ice returned by OpenIFS, so a small FESOM difference spreads through the coupling within the first day.

The standalone CI tests (pi mesh, 1 against 4 threads, now including mEVP, aEVP and split-explicit) pass, so the thread dependence sits in code our coupled configuration uses and the CI does not: CORE3 with cavities, `linfs`, TKE+IDEMIX mixing, the OASIS coupling. TKE and IDEMIX contain no OpenMP; the OpenMP sum reductions left after #1099 run at initialisation or in diagnostics. The source is not yet located. Within an experiment the FESOM thread count therefore has to stay fixed. nolockI computed 199 s in FESOM against perfI's 182 s; at one thread the gathers do more arithmetic than the lock-free-at-one-thread scatter they replace, but one run each cannot separate that from node variation.

1.21 One thread on two cores: FESOM is faster without OpenMP (2026-09-29)

nolockB4 reserved and pinned two cores per FESOM rank, exactly as nolockB1, but started FESOM through a wrapper (`bin/fesom_omp1.sh`) that sets `OMP_NUM_THREADS=1`. With the second core of

each pair idle, FESOM computed 135 s against 161 s with two threads on the same cores, and the model stepped 2.51 against 2.65 s per model day. The same 768 single-threaded ranks packed 128 per node on 6 nodes compute 228–238 s (pmcP, mevpB1t) and step 3.32–3.40 s per model day on 23 nodes: spreading costs 6 more nodes and still uses 7 % fewer node-seconds per model day (73 against 78). Figure 2(a) collects the layouts.

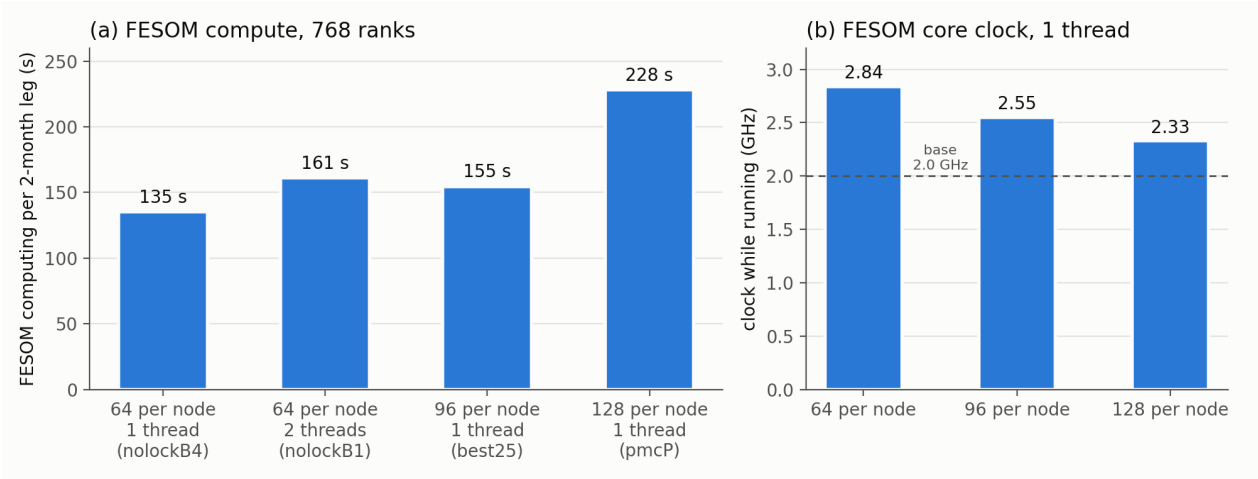


Figure 2: (a) FESOM computing time per 2-month leg (LUCIA timelines) for the same 768 ranks and binary in four layouts. The second OpenMP thread costs 19 % against leaving its core idle; packing 128 ranks per node costs 69 %. (b) Clock of the busy FESOM cores while they run, from hardware counters: the fewer cores of a node are busy, the higher the rest boost above the 2.0 GHz base clock. DRAM traffic stays at 40 to 70 GB/s per node in every layout, far below what a node delivers, so the idle cores buy clock and some cache, not bandwidth.

1.22 Hardware counters: idle neighbours buy clock, not bandwidth (2026-09-29)

Albedo has neither `perf` nor likwid, but `perf_event_paranoid`=2 allows user-mode counters on one's own processes. `bench/pmcwin.c` attaches to every thread of every FESOM and OpenIFS process on a node for 30 s once OpenIFS has passed model day 10 (started once per node by `bin/pmcwrap.sh` through `bin/pmcmon.sh`). It counts task clock, cycles, instructions and three Zen2 events for lines read from DRAM: demand L1D fills (0x43, umask 0x48), L1D hardware-prefetch fills (0x5A, umask 0x48) and L2 prefetches that miss L2 and L3 (0x72). `bench/pmc_summary.py` turns them into clock while running, instructions per cycle and DRAM traffic.

component	layout (run)	GHz	IPC	DRAM, GB/s per node
FESOM	64 per node, 1 thread (pmcS, pmcO2)	2.84	1.90	39
FESOM	96 per node, 1 thread (fes96)	2.55	1.73	66
FESOM	128 per node, 1 thread (pmcP)	2.33	1.71	≈ 70
OpenIFS	384 × 4, 4 threads (pmcS, pmcP, fes96)	2.44–2.49	1.32–1.40	19–25
OpenIFS	2 threads on 4 reserved cores (pmcO2)	2.81	1.84	21
OpenIFS	768 × 2, 2 threads on 2 cores (oifs768)	2.41	1.54	26

Table 15: Counter windows of 30 s around model day 10. The pmcP value is doubled from 64 sampled of the 128 ranks per node (a limit in the first `pmcwin`, since fixed). A single busy core on an otherwise idle node ran at 3.28 GHz.

Clock and instructions per cycle together give a spread FESOM rank 1.36 times the instruction rate of a packed one; FESOM's computing time differs by 1.65. The remainder is presumably waiting in MPI, which grows when every rank is slower; the counters cannot show it, since polling also retires instructions. OpenIFS with two threads on its four reserved cores computed as fast as

with four (177 against 175 s): its third and fourth threads contribute nothing at TCO95.

1.23 Critical path per coupling step (2026-09-29)

`bench/lucia_steps.py` splits the LUCIA timelines into the 1414 coupling intervals of a 2-month leg (one per 3600 s OpenIFS step, two FESOM steps each) and gives each component's computing and waiting per interval. In pmcS (FESOM 64 per node, 29 nodes, 2.53 s per model day, about 94 SYPD in steady state) OpenIFS is the longer component even in its cheapest intervals (88 to 100 ms against FESOM's 77 to 88), longer still in every third interval, which carries the radiation (115 to 140 ms), and in one output interval per day (about 210 ms). FESOM is longer only in its own daily output interval (158 ms). Summing the longer of the two per interval gives a critical path of 163.0 s: OpenIFS 10 or 20 % faster would shorten it by 7 or 13 %, an infinitely fast FESOM by 3 %.

1.24 More OpenIFS ranks, fewer FESOM cores (2026-09-29)

From the pmcS intervals, fewer FESOM ranks do not pay: at 640, 512 or 384 ranks FESOM would move onto the critical path and cost 6, 21 or 54 % in speed for −1, +4 or +22 % node-seconds. Fewer cores for the same 768 ranks does: a second option of the placement module, `ranks_per_node` on a component that is not interleaved, starts it on fresh nodes and spreads that many ranks evenly over each node's cores, at 96 per node three of every four, one idle core per L3 group. OpenIFS was tried at 768×2 on the same nodes as 384×4.

run	FESOM	OpenIFS	nodes	s per model day	FESOM computing	OpenIFS computing
pmcS	64 per node	384×4	29	2.53	138.1	174.8
oifs768	64 per node	768×2	29	3.02	141.9	197.7
fes96	96 per node	384×4	25	2.51	158.0	173.6
fes96o768	96 per node	768×2	25	2.81	161.1	185.2

Table 16: FESOM 768×1 throughout; XIOS four per OpenIFS node. FESOM at 96 per node runs at 2.55 GHz, computes 14 % longer and still stays shorter than OpenIFS, so the model is as fast on 4 fewer nodes. OpenIFS at 768×2 packed is slower than 384×4 (2.41 GHz, 1.54 IPC). The running experiments cannot change OpenIFS's decomposition anyway: their OpenIFS restarts are tied to it.

1.25 Production layout (2026-09-29)

The largest XIOS server grows to 44–46 GB in a 10-year leg. Four next to 28 OpenIFS ranks of about 3 GB would need about 268 GB on a 257 GB node late in a leg; two per node need about 182 GB and still fit the 29 nodes. TEST_prodlayout ran FESOM 64 per node on two reserved cores with one thread, XIOS two per OpenIFS node and rnfmap on an OpenIFS node, with esm_tools installed into the experiment's own virtual environment from GitHub, as production does: 2.39 s per model day on 29 nodes, OpenIFS computing 166 against 173 s in nolockB4 (one run each). TEST_best25, the same with FESOM at 96 ranks per node, stepped 2.40 s per model day on 25 nodes: about 99 SYPD in steady state and 6.1 node-hours per model year, against 14.5 for perfH (47 nodes, 1800 s) and 19.8 in the first albedo legs (47 nodes, 1200 s, 4.16 s per model day).

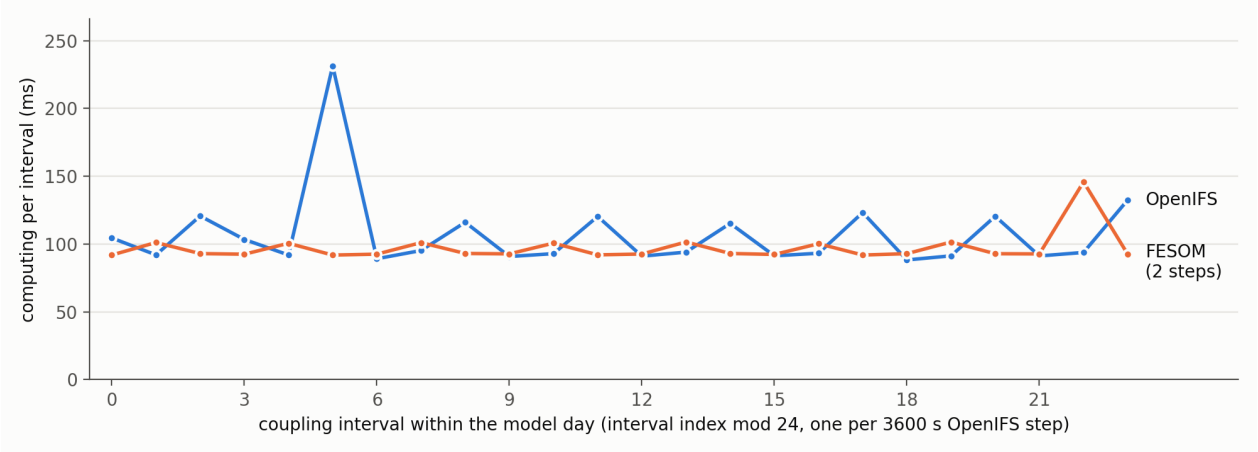


Figure 3: Computing time per coupling interval in TEST_best25, averaged over the 59 model days by position in the day. In ordinary intervals FESOM (two steps, 92–102 ms) is now as long as OpenIFS (88–95 ms); the long intervals are OpenIFS's radiation steps (every third, 115–133 ms), one OpenIFS output interval (231 ms) and one FESOM output interval (146 ms) per day. FESOM is the longer component in 823 of 1414 intervals. The critical path is 163.7 s; OpenIFS 20 % faster alone would shorten it by 8.5 %, an infinitely fast FESOM by 7 %.

Both production runscripts (PICAL_ccnice from leg 2130, PICAL_crunveg from leg 2120) now use this layout with the 53e095ec build, OpenIFS unchanged at 384 × 4. Their frozen esm_tools virtual environments (6de9ed453, without the placement module) were moved aside, so the next submission rebuilds them from `feat/awiesm3-v3.4-co2` at 322f0ed1f. They have not been resubmitted yet.

1.26 A layout for the 1200 s time step (2026-09-30)

The Labrador Sea stopped convecting after the switch to the 1800 s FESOM step, so a decade at 1200 s was prepared to separate the time step from the FESOM build. At 1200 s FESOM computes 1.37 to 1.39 times as long per model day (perfE/G against perfF/H), and 1024 instead of 768 ranks at equal density cut that by a factor 0.895 (perfJ against perfM). From the 2-month LUCIA timelines this gives FESOM about 215 s per two months at 768 ranks, 96 per node (25 nodes), 189 s at 64 per node (29 nodes), 193 s at 1024 ranks, 96 per node (28 nodes) and 169 s at 64 per node (33 nodes), against OpenIFS's 166 to 177 s. The last one balances the two. Measured over full 10-year legs:

experiment	build	FESOM step, layout	nodes	per decade	s per day
ts1200	new	1200 s, 1024 × 1, 64 per node	33	2 h 46 min	2.72
ob1200	old	1200 s, 1024 × 1, 64 per node	33	3 h 19 min	3.26
ob1800	old	1800 s, 768 × 1, 96 per node	25	3 h 12 min	3.15

Table 17: Whole 10-year legs of PICAL_crunveg_ts1200, _ob1200 and _ob1800, including start-up and restart writing; new build icehf, old build leadclose. The new build at 1200 s on 33 nodes runs a decade in the estimated 2 h 45 min (about 87 SYPD, 9.1 node-hours per model year). The old build is 20 % slower in the same layout.

1.27 REcoM: FESOM becomes 6.8 times more expensive (2026-10-03)

PICAL_cc continues the tuned physics run with the carbon cycle switched on (awiesm3 `develop-cc`: REcoM 2p3z2d, 33 FESOM tracers instead of 2, concentration-driven CO_2). Its first leg (2140–2144) ran in the 25-node physics layout and took 8 h 20 min for five years: 16.2 s per model day, 14.6 SYPD and 41.2 node-hours per model year, 6.8 times the 2.40 s of the physics-only run. FESOM's own

timers for a model month in that layout (TEST_ccA): 555 s in total, of which 364 s (66 %) is the tracer step (advection and diffusion of 33 tracers), 39 s the REcoM biology, 30 s output and 24 s sea ice. OpenIFS needs about 2.4 s per model day in this layout (physics-only run), so it spends most of the 16.2 s waiting for FESOM.

All REcoM tests below are one model month from the same 2140-01-01 start (FESOM physics restart, REcoM from climatology, OpenIFS from initial conditions) unless marked warm, on the 1800 s step. Speed is the mean of model days 2 to 30 from `ifs.stat`; tools and the full table are in `cc_perf/`.

1.28 REcoM: FESOM rank count, packing and threads (2026-10-03)

With REcoM the rank count is the lever, much more than for the physics. With OpenIFS 384 × 4 and the other components as in production:

FESOM	nodes	s per model day	SYPD	node-hours per year
768 × 1, 128 per node	23	22.24	10.6	51.9
768 × 1, 96 per node (production)	25	16.24	14.6	41.2
768 × 1, 64 per node	29	11.79	20.1	34.7
768 × 2 threads, 64 per node	29	12.07	19.6	35.5
768 × 1, 32 per node	41	8.59	27.5	35.7
768 × 4 threads, 32 per node	41	9.93	23.8	41.3
512 × 1, 64 per node	25	18.52	12.8	47.0
1024 × 1, 64 per node	33	8.66	27.3	29.0
1792 × 1, 64 per node	45	5.31	44.6	24.2
1792 × 1, 96 per node	36	6.34	37.3	23.2
1792 × 1, 128 per node	31	10.21	23.2	32.1
2688 × 1, 64 per node	56	3.93	60.2	22.3
2688 × 1, 96 per node	42	4.59	51.5	19.6
3584 × 1, 96 per node	52	3.97	59.6	20.9

Table 18: FESOM layouts with REcoM. LPJ-GUESS is 64 × 8 in the 768- and 512-rank rows and in 1024 and 1792 at 64 per node, 64 × 2 (one node instead of four, see below) in the others.

- Packing 128 ranks per node is worst (22.2 s at 768 ranks against 16.2 at 96 per node and 11.8 at 64 per node), and the tracer step scales with the ranks per node: 514, 364 and 247 s per model month at 128, 96 and 64 per node. The clock difference measured for the physics (2.33 against 2.84 GHz, 22 %) explains only a small part of that factor of 2.1; the counters were not run on the REcoM tests.
- OpenMP does not pay with REcoM either: the idle cores used as threads (768 ranks with 2 threads at 64 per node, 4 threads at 32 per node) are 3 and 16 % slower than leaving them empty. REcoM's own code has no OpenMP directives.
- More ranks are faster and cheaper together up to 2688: at 64 per node 512, 768, 1024, 1792 and 2688 ranks take 18.5, 11.8, 8.7, 5.3 and 3.9 s per model day. The tracer step goes from 247 (768) to 98 (1792) and 70 s (2688) per model month. Beyond that it flattens: 3584 ranks at 96 per node (52 nodes) are as fast as 2688 at 64 per node (56 nodes). What does not shrink is sea ice (21 to 24 s per month at every rank count), the ssh solver (6 to 7 s) and the wait for the coupling fields.
- 96 per node is the cheapest density per model year at every rank count tested.

The partitions `dist_2688` (21 × 128) and `dist_3584` (28 × 128) were cut for these tests with FESOM's `fesom_meshpart`, and the OASIS start set was reordered from the 768 order to 384, 512, 1024, 1792, 2688 and 3584 ranks (`cc_perf/reorder_cc.sh`, order checked against `nod2d.out` with `cc_perf/check_order.py`).

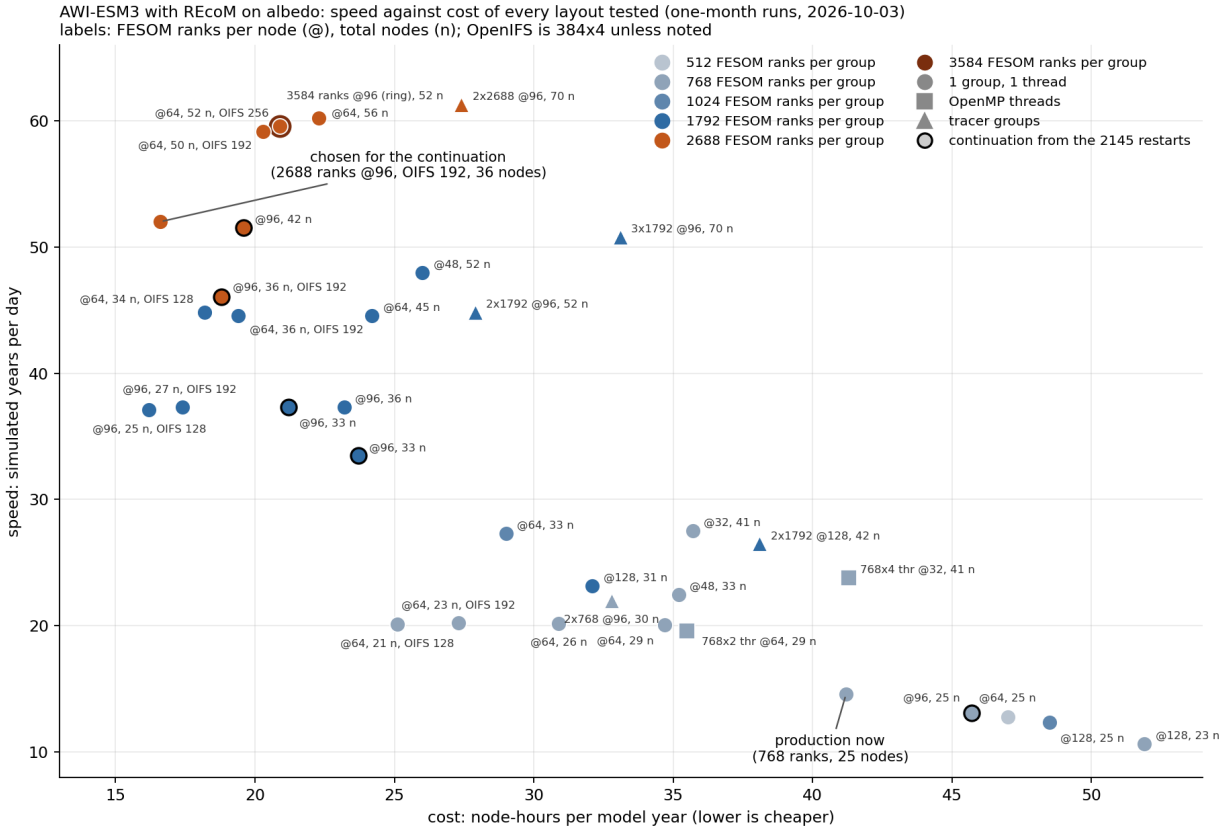


Figure 4: Speed against cost for every REcoM layout tested. Colour is the number of FESOM ranks per group, shape how the cores are used (circles one thread, squares OpenMP threads, triangles tracer groups), a black rim a warm continuation from 2145. Points move up and to the left with the rank count alone; threads and groups always sit to the right of plain ranks at the same speed. Production is at the bottom right, the layout chosen for the continuation is the orange point at 16.6 node-hours per year and its continuation test from the 2145 restarts the black-rimmed one at 18.8; the 42-node layout that keeps OpenIFS at 384×4 is the black-rimmed orange point at 19.6.

1.29 REcoM: OpenIFS and LPJ-GUESS can shrink, OpenIFS's restart cannot (2026-10-03)

With FESOM this slow, OpenIFS and LPJ-GUESS are oversized. At 768×1 (64 per node) OpenIFS 192×4 with three XIOS servers per OpenIFS node (11.69 s per model day), 128×4 with four (11.76) and LPJ-GUESS 64×2 on one node instead of 64×8 on four (11.72) are all as fast as the full set (11.79). At 1792 and 2688 ranks OpenIFS 192×4 is still not on the critical path (6.34 and 4.55 s per model day at 96 per node). XIOS peaked at 43.6 GB per server over the five-year PICAL_cc leg, so three servers per node fit next to OpenIFS for five-year legs and four do not.

OpenIFS writes one restart file per MPI task (`*.N_384`), and a run with 384×2 from PICAL_cc's 384×4 restart aborted reading it (`DDHRCV: WRONG LENGTH`, TEST_ccR1), so neither its rank count nor its thread count can change in a continuation. LPJ-GUESS's state is one file per rank too, but a change of its thread count restarted cleanly. Its state exists only at year boundaries, so a continuation test has to start on 1 January (a February restart crashed in the state reader).

Warm continuations from PICAL_cc's 2145-01-01 restarts (FESOM physics and REcoM, LPJ-GUESS, OpenIFS 384×4, coupler files reordered to the new partition) run as fast as the cold tests: 6.34 s per model day at 1792 ranks (TEST_ccR5, against 6.34 cold) and 4.59 at 2688 ranks (TEST_ccR4, against 4.59 cold). Two warm runs were 11 % slower (TEST_ccR2 7.07 at 1792 ranks, TEST_ccR3 18.03 at 768): their tracer step by 13 %, REcoM by 20 to 29 % and the sea-ice EVP by 20 to 24 %. The identical TEST_ccR5 was not, so this is the node set the job landed on, not the

warm start.

1.30 REcoM: tracer groups with XIOS output (2026-10-03)

FESOM can split its tracers between several groups of ranks (`num_fesom_groups` in `&run_config`, `fesom.mpi_num_groups` in esm_tools; each group runs the whole dynamics and a share of the tracers and broadcasts its tracers to the other groups). With the albedo build both group tests hung at start-up, group 0 inside `xios_initialize`. The colleague's group runs on levante write FESOM output without XIOS (`with_xios: false`), which is not an option here. The cause is in OASIS's group support:

- The root rank of each model is the maximum over all tasks that are the root of their local communicator. With groups every group's first task qualifies, so OASIS reported the last group's root (rank 768) as FESOM's root. Only group 0 initialises XIOS, so the XIOS server waited on group 1's root and group 0 on the server.
- `oasis_enddef` starts with a split of the global coupling communicator, which includes the groups above 0, but only group 0 calls `oasis_enddef`.

The colleague's OASIS checkout on levante has an equivalent fix as uncommitted changes (it narrows the global communicator per group and needs every component rebuilt). For the tests a smaller one went into a separate build tree (`/albedo/work/user/jstreffi/model_codes/awiesm3-develop-cc-grp/`): only group 0's root counts as the model root, and groups above 0 join the split with `MPI_UNDEFINED` through a new `oasis_enddef_inactive_group`, called by FESOM; only FESOM is relinked. Groups then run with XIOS output, but never beat plain ranks on the same nodes:

groups	nodes	s per day	SYPD	one group on similar nodes	nodes	s per day
2×768, 96 per node	30	10.76	22.0	1024×1, 64 per node	33	8.66
2×1792, 128 per node	42	8.93	26.5	2688×1, 96 per node	42	4.59
2×1792, 96 per node	52	5.28	44.8	3584×1, 96 per node	52	3.97
3×1792, 96 per node	70	4.66	50.8	2688×1, 64 per node	56	3.93
2×2688, 96 per node	70	3.86	61.3	3584×1, 96 per node	52	3.97

Table 19: Tracer groups against a single group of ranks. Groups do split the tracer step (120 to 77 s per model month from one to two groups of 1792), but each group repeats the dynamics, sea ice and output, which more ranks would have divided. Start-up took 10 to 15 min at 5000 to 5800 tasks.

1.31 REcoM: the layout for the continuation (2026-10-03)

The layout chosen for continuing PICAL_cc from 2145 uses 36 nodes: FESOM 2688×1 at 96 per node, OpenIFS 192×4 from initial conditions (its 384-rank restart cannot be read), three XIOS servers and the runoff mapper on the OpenIFS nodes, LPJ-GUESS 64×2, five-year legs of about 2.5 h. The cold test of this layout ran at 4.55 s per model day (52.0 SYPD, 16.6 node-hours per model year, 2.5 times cheaper than the first leg); the continuation test from the 2145 restarts (TEST_ccP1) ran cleanly at 5.14 s (46.0 SYPD), with the tracer step 16 % slower and sea ice unchanged, the node-set scatter above. Keeping OpenIFS at 384×4 would continue the atmosphere too, at 42 nodes and 19.6 node-hours per model year. The runscript is drafted (`awiesm3-develop-cc-albedo-TCO95L91-CORE3_PICAL_cc_36node_DRAFT.yaml`) as a new experiment branched from PICAL_cc, because esm_tools would otherwise resume OpenIFS from the 384-rank restart; it has not been submitted.

The job account is chosen per leg in the runscript rather than in esm_tools: `account: !ENV ${ESM_ACCOUNT}`, with `pre_run_commands` exporting the account with the highest fairshare from `sshare`, so each leg's job picks again before it submits the next leg. Fairshare is the only priority term that differs between one user's accounts; the REcoM tests alone moved one account from 0.88 to 0.76.

2 State of the albedo performance (2026-10-16)

Proven, with REcoM (develop-cc, 33 tracers):

- With REcoM FESOM is 6.8 times as expensive: the 25-node physics layout steps 16.2 s per model day (14.6 SYPD, 41.2 node-hours per model year); the tracer step is 66 % of FESOM's time.
- FESOM's rank count is the lever: at 96 per node 768, 1792, 2688 and 3584 ranks give 16.2, 6.3, 4.6 and 4.0 s per model day on 25, 36, 42 and 52 nodes. 96 per node is the cheapest density, 128 per node the slowest.
- OpenMP threads and tracer groups are slower than plain ranks on the same nodes; groups need an OASIS fix to run with FESOM's XIOS output.
- OpenIFS 192×4 and LPJ-GUESS 64×2 keep up with FESOM; OpenIFS's restart is tied to both its rank and thread count.
- Chosen continuation layout: 36 nodes, 4.55 to 5.14 s per model day (46 to 52 SYPD), 16.6 to 18.8 node-hours per model year.
- Identical one-month jobs differ by up to 11 % in speed, depending on the nodes.

Proven, physics only (to 2026-09-30):

- At 1200 s, FESOM 1024×1 at 64 per node on 33 nodes runs a decade in 2 h 46 min (2.72 s per model day including start-up and restarts).
- Albedo runs the same configuration 1.49 times slower than levante (15483 against 10385 s per 10-year leg).
- Albedo's cores are 16 % slower on cache-resident work; memory bandwidth per core is within 4 %.
- Compute-bound parts are 17 to 29 % slower, consistent with the slower cores.
- Communication-bound parts are 41 % (ssh solver) and 83 % (ice EVP) slower, the OASIS setup six times.
- At 1200 s FESOM limits the coupled model on albedo (LUCIA timelines: FESOM 294 s, OpenIFS 204 s computing per 2 months).
- FESOM's cost per model day doubled since August: 1200 s instead of 1800 s time step, TKE+IDEMIX instead of KPP, K_{GM} 2500 instead of 1000.
- At 1792 ranks FESOM gains only 8 % from a second OpenMP thread; at 768 ranks a second thread costs 19 % against leaving its core idle.
- The FESOM time step of 1800 s gives 72 SYPD on 47 nodes against 56 at 1200 s; both production runscripts now use 1800 s.
- No MPI setting available on albedo speeds up the model: `coll/han` and the DC random send policy help the benchmark but not FESOM, RC fails at this size, and HCOLL is not built into any albedo OpenMPI.
- Inside one node the halo exchange is twice as slow as on levante with every transport available.
- The initialisation time varies between runs (FESOM setup 14 to 335 s for identical configurations).

- CMIP7 output costs 24.5 % of OpenIFS CPU and is required.
- FESOM ranks spend 60 to 75 % of the stepping time polling in MPI and 23 to 35 % computing, on albedo and on levante alike; ice-free ranks wait longest. Output-mean accumulation costs 7 % of FESOM's wall time on both (TEST_prof).
- The same 2-month profiling run steps 1.23 times slower on albedo than on levante at 1800 s; the polling is 1.2 times slower, FESOM's own code 1.1 to 1.7 times.
- In steady state the machine factor is 1.46 at 1200 s (10-year legs) and 1.09 at 1800 s (perfH 3.03 against TEST_1800 2.77 s per model day). Over a 2-month leg it is 1.15, because the first model day (OpenIFS cold start) and the last (restart and output writing) take about twice as long on albedo.
- Levante gains only 2.5 % from the 1800 s FESOM step, albedo 25 to 30 %. At 1200 s FESOM limits on albedo (294 against 204 s computing per 2 months, LUCIA timelines). At 1800 s FESOM and OpenIFS compute equally long on both machines (albedo 209 and 209 s, levante 167 and 171 s), so a faster FESOM alone gains at most the 37 to 48 s OpenIFS waits.
- OpenIFS is 1.22 times and FESOM 1.25 times slower on albedo than on levante in computing time.
- FESOM `main` with the OpenMP performance stack (#1060 merge 1ef85546) computes 13 % less at 1792×1 and 23 % less at 1024×2; albedo then steps at 2.72 s per model day (87 SYPD steady state), as fast as levante with the old binary, and OpenIFS becomes the limiter.
- With XIOS packed four servers per node and FESOM at 768×2, the model runs on 33 instead of 47 nodes at 2.86 s per model day (83 SYPD), 26 % fewer node-hours per simulated year; OpenIFS at 192×8 is slower than 384×4.
- XIOS servers are busy at most 41 %. By memory, four fit on a node of their own, but only two next to OpenIFS ranks over a 10-year leg (44–46 GB per server).
- Components can share nodes (esm-tools/esm_tools#1603): XIOS and the runoff mapper on the OpenIFS nodes free four nodes at unchanged speed.
- FESOM runs fastest single-threaded with idle cores next to its ranks: the same 768 ranks compute 135 s per 2 months at 64 per node, 155 to 158 s at 96 and 228 to 238 s at 128. The busy cores run at 2.84, 2.55 and 2.33 GHz; DRAM traffic of 40 to 70 GB/s per node is far below what a node delivers.
- OpenIFS at TCO95 gains nothing from its third and fourth thread (two threads on four reserved cores compute as fast), and 768×2 packed is 13 % slower than 384×4.
- Production layout: 25 nodes step 2.40 s per model day, about 99 SYPD in steady state and 6.1 node-hours per model year (19.8 in the first albedo legs). OpenIFS and FESOM now share the critical path: OpenIFS 20 % faster alone would gain 8.5 %, an infinitely fast FESOM 7 %.
- After #1097–#1099 a rerun at a fixed FESOM thread count is bit-identical; a different thread count still changes the results from the first day.

Open:

- A standalone CORE3 FESOM run on both machines with identical setup would isolate the machine difference from the coupling; levante has CORE3 FESOM 2.6 runs to copy the setup from.
- Whether the production layout loses speed from spanning several leaf switches (`no-switch-enforce`); the benchmarks ran within one switch.

- Whether the intra-node halo penalty is the Zen2 core-complex layout (4 cores per L3 slice against 8 on Zen3); not tested directly.
- FESOM's timing in Miguel's 100-SYPD runs.
- Whether FESOM's polling is halo latency, ice load imbalance or the ssh solver's reductions; the sampler records no call stacks. A FESOM rank-count scan (fewer ranks, same layout otherwise) would show how far 1792 ranks are past FESOM's scaling limit.
- Why EVP on rank 1791 is 2.8 times slower on albedo; first rerun with the same FESOM binary on both machines.
- Cheaper output-mean accumulation in `io_meandata`, and a partition weighted for sea ice.
- Which code makes the coupled FESOM depend on its thread count; the standalone CI configuration does not.
- A FESOM partition cut for 96 ranks per node ($n_part = 8, 96$) instead of `dist_768`'s $(12, 64)$; it needs its own reordered OASIS set.
- Speed and XIOS memory of the 25-node layout over a full 10-year leg.
- OpenIFS's radiation steps and its daily output step now set the pace; the radiation frequency and the FullPos output are the levers on the OpenIFS side.
- REcoM: why the tracer step slows so much with dense packing (514 against 247 s per month at 128 and 64 per node); the counters were not run on these tests.
- REcoM: which nodes make a job 11 to 13 % slower, and whether excluding them is worth it.
- REcoM: XIOS memory over a 10-year leg with three servers per node; five-year legs are used until it is known.
- The OASIS group fix: the colleague's version (all components rebuilt) or the test-tree version (FESOM relinked) would have to be committed before groups are used anywhere; at present groups have no layout where they pay.

3 Falsified claims (kept, struck through)

- ~~OpenIFS computes 0.066 s and waits 0.089 s per step (LUCIA summary, TEST_perfA).~~ The summary's OpenIFS column is wrong (negative in most runs); the timelines give 186 s computing and 54 s waiting for perfA.
- ~~FESOM still limits on albedo at 1800 s.~~ Also from the LUCIA summary; the timelines show FESOM and OpenIFS computing 209 s each in perfH.
- ~~On levante FESOM does not limit at 1200 s, so the 1800 s step gains nothing there.~~ The evidence was a profiled levante run compared with unprofiled production legs. The unprofiled TEST_1800 later showed a small gain (2.5 %), with FESOM and OpenIFS balanced at 1800 s, so the conclusion was close but the argument did not hold.
- ~~FESOM computes only 84 % of the time and has slack, so its cores can be reduced.~~ The 84 % omitted FESOM's output and diagnostics categories; LUCIA shows FESOM computing 91 % and waiting 9 %.
- ~~OpenIFS computes 83 % of the time and both models are saturated.~~ The `ifs.stat` CPU column includes MPI busy-waiting; LUCIA shows OpenIFS waiting 57 % of the time for FESOM.

- ~~The initialisation is five times slower than on levante as a property of albedo.~~ The same files read in 22 s in TEST_perfA and FESOM setup ranged from 14 to 335 s in identical configurations; it varies from job to job.
- ~~Give the freed XIOS nodes to OpenIFS, 384 to 768 tasks.~~ OpenIFS scaling and stability end at about four times the truncation, 380 tasks for TCO95; measured, 768×2 on the same nodes computes 13 % longer than 384×4 (oifs768).
- ~~Tidy and file movement make the short legs slow.~~ Tidy is threaded and takes 20 s; initialisation takes 5.5 min.
- ~~A precision difference between the machines.~~ OpenIFS is single precision on both ("The model is using single precision").
- ~~Different FESOM build or namelists.~~ Namelists are identical and albedo's build targets AVX2 while levante's is generic.
- ~~The DC transport's default policy slows FESOM; `UCX_TLS=rc_x,sm,self` or `UCX_DC_MLX5_TX_POLICY=rand` fixes it.~~ Both speed up the benchmark by 2 to 7 times, but RC fails at 2256 ranks and the DC random policy leaves FESOM's EVP and ssh solver unchanged (TEST_perfG/H).
- ~~Hierarchical collectives (`coll/han`) close the allreduce gap for the ssh solver.~~ Benchmark allreduce 53 to 24 μs, model unchanged (TEST_perfE/F).
- ~~Kernel mitigations make intra-node exchange slow through system calls in CMA/knem.~~ Transports without system calls (posix, sysv, vader) are as slow or slower than the default.
- ~~Albedo's cores are 1.5 to 1.8 times slower on cache-resident code.~~ Measured 16 %.
- ~~A GCC build or missing optimisation flags (FESOM PR #685).~~ Both machines build with Intel ifort; albedo has more aggressive flags than levante.
- ~~FESOM on albedo is limited by memory bandwidth, so spreading its ranks helps by giving each rank more bandwidth.~~ Hardware counters show 40 to 70 GB/s of DRAM traffic per node in every layout, far below the node's bandwidth; spread ranks win through clock (2.84 against 2.33 GHz) and instructions per cycle (1.90 against 1.71).
- ~~Removing the OpenMP locks (#1097–#1099) makes the coupled model independent of the FESOM thread count.~~ Reruns become bit-identical, but 768×2 and 768×1 on the same cores still differ from day 1 (nolockB1, nolockB4).
- ~~FESOM's tracer groups cannot be used with FESOM's XIOS output.~~ They hung in the albedo build, but because of OASIS's choice of the model root and its `oasis_enddef` split; with those fixed they run with XIOS output (TEST_ccG1–G5).
- ~~A warm REcoM continuation is about 11 % slower than a cold start.~~ Seen in TEST_ccR2 and R3; the identical TEST_ccR5 and the 2688-rank TEST_ccR4 ran exactly as fast as their cold counterparts. Node-set scatter.
- ~~The lock-free gathers reproduce the old scatter bit for bit at one thread.~~ The summation order per node is the same by construction, but perfI and nolockI (1792×1, old and new build) differ from day 1 in the coupled model; compiler contraction of multiply-adds is one possible cause, not yet checked.

4 Method notes

- OpenIFS wall per step from time stamp differences in `ifs.stat`; CPU per step is column 5.
- OpenIFS routine shares from the timing statistics at the end of `NODE.001_01`.
- FESOM: the `MODEL RUNTIME per task` and `runtime setup` blocks of rank 0 in the compute log.
- XIOS: `Performance report : Ratio` at the end of `xios_server*.out` and `xios_client*.out`.
- LUCIA is switched on by `oasis3mct.use_lucia: true`, which writes `$NLOGPRT 1 0 1` for OASIS3-MCT 5. The result is `load_balancing_info.txt` and `timeline_*.nc` in the work directory; `oasis/util/load_balancing/pyLucia.py` plots the timelines.
- The CPU column of `ifs.stat` includes MPI busy-waiting and must not be read as OpenIFS's compute share.
- Short tests on albedo: `compute_time` of 30 min or less puts the job into the `30min` QOS (priority 1 instead of 0).
- Standalone runs that set `--cpu_bind=none` with several OpenMP threads per rank and no `taskset` leave ranks and threads unpinned; the FESOM 2.8 release tests on 18 nodes collapsed that way (ssh solver 40 to 77 times slower than on one pinned node).
- Profiling: on levante the sampler's timer signals during start-up broke XIOS's collective `nc_create_par` for the interpolation weights ("Permission denied" on all FESOM ranks, job 27761837); `PCS_DELAY=60` starts the timer after the files are written. `PCS_ENABLE=1` and `LD_PRELOAD=libpcsampler.so` on selected ranks; `pcs_report.py <base> [top] [t0 t1]` restricts to an epoch window. Symbols of stripped libraries (UCX) are only the nearest exported name.
- Benchmark: `bench/mpibench.c`; job scripts `bench/bench_*.slurm`; the `HALO_BYTES` variable sets the halo message size.
- Seconds per model day: `bench/ifsstat_days.awk` on `ifs.stat`; steady state is the mean of days 2 to 58, since day 1 carries the OpenIFS cold start and the last day the restart writing.
- Per coupling interval: `bench/lucia_steps.py timeline_OpenIFS.nc timeline_fesom.nc`; the critical path is the sum over intervals of the longer component.
- Hardware counters: `bench/pmcwin.c`, built with `gcc -O2`, attached by `bin/pmcmon.sh`, which the wrapper `bin/pmcwrap.sh` (arguments: thread count, program) starts once per node; summarised by `bench/pmc_summary.py` with the work directory as argument. User mode only, so MPI polling in the kernel is not counted but polling in user space is.
- Fewer threads than reserved cores: set `omp_num_threads` to the cores to reserve and start the program through a wrapper that exports a lower `OMP_NUM_THREADS` (through FESOM's `execution_command`, because `bin_in_work` names the staged binary after `executable`). With `ranks_per_node` the wrapper is no longer needed for FESOM.
- Bit-comparison of runs: daily FESOM SST, sea-ice concentration and velocity and SSH, compared day by day with `numpy.array_equal`.

5 Assets

- Runtime: `/albedo/work/projects/p_awiesm3_cmip7/jstreffi/runtime/awiesm3-v3.4/`, experiments PICAL_ccnice, PICAL_crunveg, TEST_xiosbus, TEST_perfA–N, TEST_placeA/B, TEST_mevp*, TEST_nolock*, TEST_pmc*, TEST_oifs768, TEST_fes96*, TEST_prodlayout, TEST_best25, TEST_prof, TEST_xios1d; failed attempts kept as `*_rc_failed`, `*_knobfail`, `*_vtune_ptrace`.
- Benchmarks, profiler, counter and LUCIA tools, figure script: `bench/` in this repository, `https://github.com/JanStreffing/investigation_awiesm3_albedo_performance`; levante copies in `/scratch/a/a270092/lr_probe/`.
- Wrappers in `/albedo/work/user/jstreffi/model_codes/awiesm3-develop/bin/`: `fesom_prof.sh`, `libpcsampler.so`, `fesom_omp1.sh`, `pmcwrap.sh`, `pmcmon.sh`, `pmcwin`.
- FESOM builds in `/albedo/work/user/jstreffi/model_codes/awiesm3-develop/fesom-bin/`: `main1060_1ef85546` and `main1060_53e095ec` (production, libfesom.so md5 18087062).
- Partitions `dist_384`, `dist_768`, `dist_1024` in `/albedo/work/projects/p_awiesm3_cmip7/jstreffi/input/fesom2/core3/`; reordered OASIS sets in `/albedo/work/projects/p_awiesm3_cmip7/jstreffi/input/oasis3mct_reorder/`.
- esm_tools placement: `src/esm_runscripts/placement.py` on branch `feat/awiesm3-v3.4-co2` (322f0ed1f) and esm-tools/esm_tools#1603.
- Runscripts: `esm_tools/runscripts/awiesm3/develop/awiesm3-develop-albedo-TCO95L91-CORE3_*.yaml` (not committed).
- Levante reference: `/work/bb1469/a270092/runtime/awiesm3-v3.4/PICAL_ccnice/`.
- REcoM tests: `cc_perf/` in this repository: `make_cc_test.py` (one-month test runscripts from the PICAL_cc runscript), `cc_eval.sh` (speed and FESOM timers per test), `results_20261003.txt` (all tests), `plot_sypd_cost.py` (Figure 4), `reorder_cc.sh` and `check_order.py` (OASIS sets per partition). Experiments TEST_ccA–Z4, TEST_ccR1–R5, TEST_ccG1–G5, TEST_ccP1 in the runtime directory (logs and output; the run folders have been removed).
- REcoM partitions `dist_2688`, `dist_3584` next to the others; OASIS sets `/albedo/work/projects/p_awiesm3_cmip7/jstreffi/input/oasis_ini/cc_from_gmhemi1800_2139_dist*/` (2140 start) and `cc_PICAL_cc_21441231_dist{1792,2688}/` (2145 continuation).
- Tracer-group build with the OASIS fix: `/albedo/work/user/jstreffi/model_codes/awiesm3-develop-cc-grp/` (not committed).